
UNIVERSITY OF BUEA

Department of Computer Engineering

CEF 440

Internet and Mobile Programming

TASK 3: REQUIREMENT ANALYSIS

**Project: Design and Implementation of a Mobile Application
for Car Fault Diagnosis**

Group 12 Project Report Task 3

Supervisor: **Dr. Valery Nkemeni**

Academic Year: 2025 – 2026

1. Introduction

This report presents the results of the requirement analysis carried out for an AI-based car diagnostic mobile application. The system is designed to help drivers and mechanics in Cameroon identify car faults, understand warning lights, track maintenance history, and locate nearby mechanics, all from a mobile device.

The analysis draws from a survey of 80 respondents, insights gathered from the team, and a review of what works and what does not in existing diagnostic tools. This document covers the completeness and clarity of the requirements, technical feasibility, inconsistencies identified, missing information, requirement prioritization, functional and non-functional classifications, the Software Requirement Specification (SRS), and stakeholder validation.

2. Survey Overview

A survey was conducted to understand the experiences of real users with car faults and existing diagnostic tools. A total of 80 responses were collected from drivers, car owners, and people who interact with vehicles regularly. The data gathered informed the requirements for this system.

2.1 Key Findings from the Survey

Finding	Detail
Most respondents found car faults difficult to understand	A large portion rated understanding car faults as "difficult" or "very difficult"
The majority go to a mechanic when a warning light appears	Most users do not try to diagnose the fault themselves
Very few use any diagnostic tools	Most respondents said they do not use any existing diagnostic apps or tools
Almost everyone would use or consider using the app	A majority answered "Yes" or "Maybe" to using a mobile diagnostic app
The most requested features were fault detection and suggested solutions	These were the most commonly selected features across all responses
Language barrier with mechanics was frequently mentioned	Several users listed this as one of the main challenges they face
Offline mode was one of the most repeated suggestions	Many respondents specifically asked for the app to work without an internet connection
Common faults reported include transmission problems, brake pad wear, and battery issues	These were the most frequently mentioned fault types in open responses

Common challenges respondents faced included not knowing the exact fault, confusing error messages, high repair costs, language barriers with mechanics, spare parts unavailability, and getting overcharged. These pain points directly shaped the requirements for the system.

3. Requirement Analysis

This section reviews the requirements gathered during the elicitation phase. They were evaluated based on completeness, clarity, technical feasibility, and dependency relationships.

3.1 Completeness

The requirements cover the core features needed for the system to work: fault detection through OBD-II integration, image and sound-based AI diagnosis, bilingual support, maintenance history storage, nearby mechanic suggestions, cost estimation, and user alerts. The system also includes non-functional requirements such as response time, offline mode, and data security.

However, a few areas were found to be incomplete during review:

- No clear requirement exists for how the system should handle faults it cannot recognize or does not have data on.
- The handling of noisy environments during audio-based fault detection was not addressed.
- There is no specification for what happens when the app loses connectivity mid-diagnosis.
- Requirements for how the system should behave on low-end or older smartphones are missing.

3.2 Clarity

Several requirements were initially vague and needed to be made more specific. For example:

- "The app should detect engine sounds" was refined to: the app should be able to identify at least five common abnormal engine sounds with a minimum accuracy of 85%.
- "Fast image processing" was defined as: the system should return a diagnosis from an uploaded photo within 10 seconds under normal network conditions.
- "Support multiple languages" was clarified to mean English and French as the two primary languages, with future support for Pidgin English as a local variant.

Making these requirements measurable is important because it sets a clear standard that developers can build to and testers can verify against.

3.3 Technical Feasibility

Most of the proposed features are technically achievable using current technology. OBD-II integration is a well-established standard supported by most modern vehicles manufactured after

1996. AI-based image and sound analysis are possible through mobile machine learning frameworks. Cloud storage for maintenance history is straightforward to implement.

That said, a few areas require careful consideration:

- AI accuracy may vary depending on the phone camera quality, lighting conditions, and the background noise level during sound analysis.
- Full offline functionality for AI diagnosis is challenging, as some AI models require server-side processing. A hybrid approach where basic checks work offline and advanced analysis requires connectivity is more realistic.
- Building a reliable database of vehicle faults specific to car brands commonly found in Cameroon, such as Toyota, Nissan, and Mitsubishi, will require significant data collection effort.
- Real-time mechanic chat would depend on mechanic availability and willingness to use the platform, which is outside the system's direct control.

3.4 Dependency Relationships

Several features depend on other features being in place first. Understanding these dependencies helps with planning development in the right order.

Feature	Depends On
Fault detection via AI	OBD-II connection or camera/microphone access
Maintenance history	User account and cloud storage setup
Cost estimation	Fault detection results and a local pricing database
Nearby mechanic suggestions	Location permission and an up-to-date mechanic directory
Predictive maintenance alerts	Maintenance history records and AI analysis engine
Bilingual interface	Complete translations and language selection at onboarding
Offline mode	Local storage and a cached version of the fault database

4. Inconsistencies, Ambiguities, and Missing Information

4.1 Inconsistencies

Issue	Description	Suggested Resolution
Offline vs Online functionality	Some requirements assume constant internet access while others specify offline support	Define a tiered approach: core features work offline, advanced AI analysis requires connection
AI accuracy on different devices	The system is expected to perform consistently, but smartphone hardware varies significantly	Set minimum hardware requirements and test on low-end and high-end devices

Issue	Description	Suggested Resolution
Safety while driving	Some features like real-time audio monitoring may distract the driver	Limit active diagnosis features to when the car is stationary; add safety warnings
Technical vs non-technical users	The app is meant for both mechanics (who want technical details) and everyday drivers (who need simple language)	Implement two display modes: a simple mode and an expert mode the user can switch between

4.2 Ambiguities

- It is not clear whether the cost estimation feature should provide exact prices or general price ranges. Given that prices vary by location and mechanic, ranges with a local average are more realistic.
- "Nearby mechanic suggestions" does not define what counts as nearby. A maximum radius should be specified and made adjustable by the user.
- The requirement for OBD-II integration does not specify whether the app will support all OBD-II adapters or only specific certified ones. This affects compatibility and testing.

4.3 Missing Information

- There is no requirement for what happens when the AI encounters an unknown fault. The system should display a message telling the user the fault could not be identified and suggest they visit a mechanic.
- The survey surfaced the issue of language barriers with mechanics, but no requirement was written for an in-app translation or communication tool to help users talk to mechanics.
- There is no requirement defining data retention policies, for example how long maintenance history is kept and whether users can export or delete their data.
- No requirement exists for how the system should handle updates to the fault database, which will need regular maintenance as new vehicles and fault patterns emerge.

5. Requirement Prioritization

Requirements were prioritized using the MoSCoW method, which groups features into four categories: Must-Have, Should-Have, Could-Have, and Won't-Have for this version. This helps the team focus on what matters most in the first release.

Priority	Feature	Reason
Must Have	Fault detection (OBD-II or AI image/sound)	Core purpose of the app

Priority	Feature	Reason
Must Have	Explanation of warning lights in plain language	Requested by majority of respondents
Must Have	Step-by-step guidance for common faults	Helps non-technical users act on results
Must Have	Bilingual support (English and French)	Essential for Cameroon context
Must Have	Data security and user authentication	Protects user data and builds trust
Should Have	Maintenance history and reminders	Helps prevent repeated faults
Should Have	Nearby mechanic suggestions	Directly addresses a main user complaint
Should Have	Cost estimation for repairs	Helps users avoid being overcharged
Should Have	Offline mode for basic features	Many users specifically asked for this
Could Have	Predictive maintenance alerts based on history	Valuable but requires significant data first
Could Have	Fuel efficiency tips	Nice addition but not core to diagnosis
Could Have	Community forum	Useful long-term but not essential at launch
Could Have	Voice assistant integration	Accessibility feature for future version
Won't Have (v1)	Real-time mechanic chat	Requires a separate platform and workforce
Won't Have (v1)	Insurance claim assistance	Out of scope for this version
Won't Have (v1)	Full offline AI diagnosis	Too resource-heavy for initial release

6. Requirement Classification

6.1 Functional Requirements

Functional requirements describe what the system must do. These are the actions and behaviors the user can directly interact with.

- The system must connect to a vehicle's OBD-II port via a compatible Bluetooth or Wi-Fi adapter and read fault codes.
- The system must allow users to take a photo of a dashboard warning light or car component and return a diagnosis.
- The system must detect and classify abnormal engine sounds captured through the device microphone.
- The system must explain detected faults and warning lights in plain, non-technical language.

- The system must provide step-by-step guidance on what the user should do after a fault is detected.
- The system must suggest nearby mechanics based on the user's current GPS location.
- The system must store and display the user's vehicle maintenance history.
- The system must send maintenance reminders based on the vehicle's service schedule.
- The system must estimate repair costs based on the fault detected and the user's location.
- The system must support English and French as display languages, selectable during onboarding.
- The system must allow users to create accounts and manage their vehicle profiles.

6.2 Non-Functional Requirements

Non-functional requirements describe how the system performs rather than what it does. They define standards the system must meet in areas like speed, security, and usability.

Category	Requirement
Performance	The system must return a fault diagnosis from an image within 10 seconds under a standard 4G connection
Performance	The app must load within 3 seconds on a device with at least 2GB RAM
Reliability	The system must be available at least 99% of the time during normal operating hours
Usability	A first-time user with no technical knowledge should be able to complete a basic fault check within 3 minutes without guidance
Usability	The interface must follow a simple design that works for both young and older users
Security	All user data including vehicle history and location must be encrypted both in transit and at rest
Security	User authentication must include a strong password requirement and optional two-factor authentication
Compatibility	The app must run on Android 8.0 and above, and iOS 13 and above
Compatibility	The app must support standard OBD-II Bluetooth and Wi-Fi adapters
Offline support	Core features including warning light lookup and basic fault explanations must be accessible without internet
Localization	The fault database must include vehicles commonly used in Cameroon, especially Toyota, Nissan, and Mitsubishi models
Maintainability	The fault database must be updateable remotely without requiring a full app update

7. Software Requirement Specification (SRS)

7.1 System Purpose

The AI-based Car Diagnostic System is a mobile application that enables users to detect, understand, and respond to vehicle faults using AI-powered tools. It is designed for drivers, car owners, and mechanics operating in Cameroon, where access to formal diagnostic equipment is limited and language barriers often complicate communication between users and mechanics.

7.2 Intended Users

- Everyday drivers who are not technically trained but want to understand what is wrong with their vehicle.
- Car owners who want to track maintenance history and receive service reminders.
- Mechanics who want a digital tool to assist with diagnosis and documentation.
- Fleet managers responsible for multiple vehicles and needing a centralized maintenance overview.

7.3 System Scope

The system covers fault detection, maintenance tracking, mechanic discovery, and cost awareness. It does not cover actual vehicle repairs, direct mechanic booking, or insurance processing. The system works as a decision-support tool and information provider, not a replacement for professional mechanical assessment.

7.4 System Interfaces

- OBD-II interface: The app connects to the vehicle's ECU through a standard OBD-II Bluetooth or Wi-Fi dongle to read diagnostic trouble codes (DTCs).
- Camera interface: The mobile camera is used to capture images of warning lights or visible vehicle components for AI-based analysis.
- Microphone interface: The device microphone captures engine or mechanical sounds to support audio-based fault detection.
- GPS interface: Location services are used to find nearby mechanics and provide location-based cost estimates.
- Cloud storage: Maintenance records and user data are stored securely on a remote server with encrypted communication.

7.5 Data Management and Security

User data including vehicle profiles, maintenance history, and location information must be stored securely. The system must:

- Encrypt all data in transit using HTTPS and TLS.
- Encrypt all locally stored data using device-level encryption.
- Allow users to delete their account and all associated data at any time.

-
- Not share user data with third parties without explicit consent.
 - Store data on servers that comply with basic data protection standards.

7.6 Error Handling

The system must handle errors gracefully without crashing or losing user data. Specific error scenarios include:

- If the OBD-II connection fails, the system should notify the user and offer alternative diagnosis options such as image or sound-based checking.
- If the AI cannot identify a fault from an image or sound, it must inform the user clearly and recommend a mechanic visit.
- If internet connectivity is lost during an operation, the system should save the session and allow the user to resume when connectivity returns.
- If location services are unavailable, the system should still function without the mechanic suggestion feature and inform the user why that feature is not showing.

8. Stakeholder Validation

To ensure the requirements reflect actual user needs, the findings were reviewed against feedback from three main groups of stakeholders:

8.1 Drivers and Everyday Car Users

The survey data shows that most drivers find it hard to understand car faults and rely on mechanics without knowing whether the advice they receive is accurate. This group strongly values simple explanations, step-by-step guidance, and an affordable or free tool. Their feedback confirms the need for a clean interface, offline access, and non-technical language throughout the app.

8.2 Mechanics

Mechanics are expected to benefit from a tool that helps them document faults, retrieve maintenance history, and access technical fault details. Their feedback emphasizes the need for OBD-II integration and accurate fault codes. They prefer the technical display mode over simplified descriptions. They also noted that a locally relevant fault database is critical because generic international databases often miss fault patterns common to older vehicles and brands sold in Cameroon.

8.3 Fleet Managers

Fleet managers who oversee multiple vehicles need a view of all vehicles under their management, including maintenance schedules and outstanding faults. Their feedback points to the importance of maintenance reminders, service history records, and cost tracking. For this

group, the ability to export maintenance reports and receive alerts for multiple vehicles at once would be valuable additions to consider in future versions.

8.4 Validation Summary

Stakeholder Group	Key Concerns Confirmed	Requirements Validated
Drivers	Simple language, affordable access, offline mode	Fault explanation, step-by-step guidance, bilingual support, offline access
Mechanics	Accurate fault codes, local vehicle database, technical detail	OBD-II integration, local fault database, expert display mode
Fleet Managers	Multi-vehicle tracking, maintenance records, cost visibility	Maintenance history, service reminders, cost estimation

9. Conclusion

The requirement analysis carried out for this system confirms that there is a genuine need for an accessible, AI-powered car diagnostic tool designed with the Cameroonian context in mind. The survey data, combined with feedback from the team and stakeholder perspectives, has helped produce a clear, prioritized, and realistic set of requirements.

The most important things to build first are fault detection, warning light explanation, step-by-step guidance, bilingual support, and basic security. These are the features that directly address the problems users face today. Other features like predictive maintenance and community tools can follow in later versions once the core system is stable.

The main risks to manage are offline functionality, AI accuracy on low-end devices, and the quality of the local vehicle fault database. These will require focused attention during development and testing.

This requirement analysis provides the foundation for the next phase of the project, which involves system design and architecture planning based on the validated and prioritized requirements documented here.